

**INSTITUTO FEDERAL DA BAHIA
CAMPUS FEIRA DE SANTANA
BACHARELADO EM SISTEMAS DE INFORMAÇÃO**

GUSTAVO DA SILVA FERREIRA

**PERCEPÇÕES E ADOÇÃO DE TYPESCRIPT E JAVASCRIPT NO
DESENVOLVIMENTO FRONT-END: UM ESTUDO ENTRE INICIANTE E
PROFISSIONAIS EXPERIENTES**

**FEIRA DE SANTANA-BA
2025**

GUSTAVO DA SILVA FERREIRA

**PERCEPÇÕES E ADOÇÃO DE TYPESCRIPT E JAVASCRIPT NO
DESENVOLVIMENTO FRONT-END: UM ESTUDO ENTRE INICIANTES E
PROFISSIONAIS EXPERIENTES**

Trabalho de Conclusão de Curso submetido ao Curso de Bacharelado em Sistemas de Informação, do campus de Feira de Santana do Instituto Federal da Bahia (IFBA), como um dos requisitos para a obtenção do título de Bacharel em Sistemas de Informação

Orientador: Prof Dr.Clerber Jorge Lira

**FEIRA DE SANTANA-BA
2025**

Percepções e Adoção de TypeScript e JavaScript no Desenvolvimento Front-End: Um Estudo entre Iniciantes e Profissionais Experientes

Resumo

O desenvolvimento de aplicações front-end modernas demanda ecossistemas que equilibrem agilidade de entrega, facilidade de manutenção e confiabilidade de código. Nesse cenário, o debate entre JavaScript (JS), com tipagem dinâmica, e TypeScript (TS), com tipagem estática opcional, tornou-se relevante na engenharia de software. Este Trabalho de Conclusão de Curso investiga percepções e fatores associados à adoção dessas linguagens, comparando desenvolvedores iniciantes e profissionais experientes. Foi realizada uma pesquisa de métodos mistos, com questionário aplicado por conveniência a 23 desenvolvedores. Os dados foram analisados por estatística descritiva e categorização temática de respostas abertas. Na amostra analisada, iniciantes atribuíram maior concordância à rapidez de prototipagem em JavaScript (média 4,29), enquanto experientes relataram maior produtividade com TypeScript (4,22, contra 2,93 entre iniciantes) e maior preferência pela linguagem em novos projetos (4,56). Os participantes também associaram o TypeScript à detecção precoce de erros (4,17 no total), à manutenibilidade (4,44 entre experientes) e à padronização em equipes (4,30). Os relatos qualitativos, contudo, apontaram custos de configuração, dificuldades de migração, contratos de bibliotecas e limites da verificação estática em tempo de execução. Os resultados sugerem que a escolha entre as linguagens envolve uma decisão contextual, relacionada à escala do projeto, ao horizonte de manutenção e à experiência da equipe, sem permitir generalizações para toda a comunidade de desenvolvedores.

Palavras-chave: JavaScript. TypeScript. Desenvolvimento Front-End. Tipagem Estática. Manutenibilidade de Software. Engenharia de Software.

Perceptions and Adoption of TypeScript and JavaScript in Front-End Development: A Study
Among Beginners and Experienced Professionals

Abstract

The development of modern front-end applications demands ecosystems that balance delivery agility, maintainability, and code reliability. This study investigates perceptions and factors associated with the adoption of JavaScript (JS), characterized by dynamic typing, and TypeScript (TS), which provides optional static typing, comparing novice and experienced developers. A mixed-methods survey was conducted with a convenience sample of 23 front-end developers. Data were analyzed using descriptive statistics and thematic categorization of open-ended responses. In the sample, novices expressed stronger agreement regarding JavaScript's rapid prototyping (mean 4.29), whereas experienced participants reported greater productivity with TypeScript (4.22, compared with 2.93 among novices) and a stronger preference for TypeScript in new projects (4.56). Participants also associated TypeScript with early bug detection (4.17 overall), maintainability (4.44 among experienced participants), and team standardization (4.30). Qualitative accounts, however, identified configuration costs, migration difficulties, library contracts, and the limits of static checking at runtime. The findings suggest that language choice is contextual and related to project scale, maintenance horizon, and team experience; they do not support generalizations to the entire developer community.

Keywords: JavaScript. TypeScript. Front-End Development. Static Typing. Software Maintainability. Software Engineering.

Lista de figuras

Figura 1 – Perfil dos respondentes por tempo de experiência e participação em grandes projetos	15
Figura 2 – Comparativo de proficiência técnica autoavaliada em JavaScript e TypeScript	16
Figura 3 – Comparativo de médias no Bloco A entre Iniciantes e Experientes	17
Figura 4 – Comparativo de médias no Bloco B entre Iniciantes e Experientes	19
Figura 5 – Comparativo de médias no Bloco C entre Iniciantes e Experientes	21
Figura 6 – Comparativo de médias no Bloco D entre Iniciantes e Experientes	22
Figura 7 – Divergência perceptual consolidada entre Experientes e Iniciantes (Δ de Médias)	23

Lista de tabelas

Tabela 1 – Estatísticas descritivas do Bloco A: Produtividade e Desenvolvimento . . .	17
Tabela 2 – Estatísticas descritivas do Bloco B: Qualidade e Manutenção do Código . .	19
Tabela 3 – Estatísticas descritivas do Bloco C: Compatibilidade e Ferramentas	21
Tabela 4 – Estatísticas descritivas do Bloco D: Preferências e Perspectivas Pessoais . .	22
Tabela 5 – Cronograma de atividades do TCC	30

Lista de quadros

Quadro 1 – Confronto entre a literatura acadêmica e as evidências empíricas da pesquisa 26

Sumário

1 – Introdução	1
1.1 Justificativa	2
1.2 Objetivos	2
1.2.1 Objetivo Geral	2
1.2.2 Objetivos Específicos	2
2 – Fundamentação Teórica	3
2.1 JavaScript, tipagem dinâmica e desenvolvimento web	3
2.2 TypeScript como sistema de tipagem gradual	4
2.3 Qualidade de software, manutenção e dívida técnica	5
2.3.1 Dívida técnica como decisão de gestão	6
2.4 Contratos externos, integração e arquitetura	6
2.5 Configuração, ferramentas e migração	7
2.6 Adoção, colaboração e maturidade da equipe	8
3 – Metodologia	9
3.1 Problema de pesquisa e questões investigativas	9
3.2 Instrumento de coleta	10
3.3 População, amostragem e grupos de análise	10
3.4 Procedimentos de coleta e tratamento dos dados	11
3.5 Plano de análise quantitativa	11
3.6 Plano de análise qualitativa	12
3.7 Validade, limitações e aspectos éticos	12
3.8 Síntese metodológica	13
4 – Análise e Discussão dos Resultados	14
4.1 Caracterização Demográfica dos Participantes	14
4.2 Produtividade e Desenvolvimento Ágil (Bloco A)	16
4.3 Qualidade do Código, Manutenibilidade e Prevenção de Falhas (Bloco B)	18
4.4 Compatibilidade, Ferramentas e Desafios de Migração (Bloco C)	20
4.5 Preferências Pessoais, Confiança e Perspectivas Futuras (Bloco D)	22
4.6 Análise Qualitativa dos Relatos Práticos da Indústria	24
4.6.1 Eixo 1: Blindagem de Contratos de Dados e Integração de APIs	24
4.6.2 Eixo 2: Refatoração em Larga Escala e Propagação de Modificações	24
4.6.3 Eixo 3: Complexidade Inicial e Fricção com o Ecossistema Externo	25

4.6.4	Eixo 4: A Fronteira entre Compilação e Execução (Runtime vs Compile-time)	25
4.7	Confronto Teórico-Empírico e Síntese da Discussão	26
5	– Considerações Finais	27
5.1	Síntese dos Resultados e Atendimento aos Objetivos	27
5.2	Contribuições e Recomendações Práticas	28
5.2.1	Diretrizes para a Indústria e Gestores de Tecnologia	28
5.2.2	Diretrizes para o Ensino Acadêmico de Computação	28
5.3	Limitações da Pesquisa	28
5.4	Sugestões para Trabalhos Futuros	29
	Referências	31

1 Introdução

O desenvolvimento de aplicações para a Web representa, nas últimas décadas, um dos setores mais dinâmicos e influentes da tecnologia da informação. A crescente demanda por experiências digitais ricas, interativas e de alto desempenho impulsionou uma rápida evolução das tecnologias de front-end, a camada da aplicação com a qual o usuário final interage diretamente. Nesse ecossistema, a linguagem JavaScript (JS) se consolidou como pilar tecnológico, sendo, por mais de uma década, a mais utilizada por desenvolvedores em todo o mundo ([Stack Overflow, 2023](#)).

Originalmente concebido para adicionar interatividade a páginas estáticas, o JavaScript evoluiu, por meio da padronização do *ECMAScript*, para uma linguagem versátil e poderosa. Contudo, a mesma flexibilidade que impulsionou sua popularidade — notadamente a tipagem dinâmica — pode ampliar a distância entre o código produzido e os problemas observados em execução. Estudos sobre defeitos em JavaScript indicam que uma parcela desses problemas poderia ser detectada por verificações estáticas, embora isso não esgote as causas de falhas em aplicações web ([Makimov, 2022](#); [Richards](#); [Nardelli](#); [Vitek, 2017](#)).

Em resposta a essa lacuna, a Microsoft lançou, em 2012, o TypeScript (TS), um superconjunto de JavaScript que adiciona tipagem estática gradual e é posteriormente transformado em JavaScript. Essa arquitetura favorece a migração incremental e amplia o suporte do editor a navegação, autocompletar e refatoração, mas não constitui uma garantia de correção em tempo de execução. A literatura mais recente sugere, inclusive, que o efeito da linguagem depende da configuração do compilador e da qualidade dos contratos mantidos nas bibliotecas externas ([George, 2020](#); [Zandonotto et al., 2026](#); [Feldthaus](#); [Møller, 2014](#)).

Assim, o TypeScript não deve ser compreendido como uma solução que elimina bugs, mas como uma tecnologia que altera o perfil dos riscos. A análise de defeitos da própria linguagem identificou problemas envolvendo tipos, semântica, escopos, recursos e comportamento do compilador ([Wang](#); [Fang](#); [Wang, 2025](#)). Em projetos reais, falhas de configuração, uso de APIs, dependências e assincronicidade também podem superar os erros locais de lógica, deslocando parte da fragilidade para as fronteiras entre código, ferramentas e ambiente de execução ([Tang](#); [Alimadadi](#); [Sumner, 2026](#)).

A escolha entre JavaScript e TypeScript envolve um balanço complexo de vantagens e desvantagens, percebidas de maneiras distintas por desenvolvedores com diferentes níveis de experiência. Enquanto profissionais experientes tendem a valorizar a robustez e a estrutura que o TypeScript oferece, iniciantes podem ser atraídos pela simplicidade e pela curva de aprendizado mais suave do JavaScript ([Shrestha et al., 2022](#)).

1.1 Justificativa

Este estudo se justifica pela sua dupla relevância, prática e acadêmica. No contexto profissional, uma compreensão aprofundada dos fatores que influenciam a adoção de cada linguagem pode subsidiar decisões estratégicas de equipes e gestores de tecnologia. A escolha da ferramenta adequada impacta diretamente a produtividade, a manutenibilidade de projetos e a capacidade de escalar sistemas de forma sustentável.

Do ponto de vista acadêmico, embora a popularidade do TypeScript e os possíveis ganhos da tipagem estática sejam bem documentados, ainda é necessário relacionar esses ganhos às condições concretas de adoção. A configuração do compilador, a correspondência entre interfaces e bibliotecas JavaScript e a complexidade do toolchain influenciam o resultado obtido. Há também uma lacuna sobre como a percepção desses benefícios e desafios é modulada pelo nível de experiência do desenvolvedor, especialmente no cenário tecnológico brasileiro. Esta pesquisa busca contribuir para esse debate, examinando a escolha entre JavaScript e TypeScript a partir da literatura e das percepções de desenvolvedores.

1.2 Objetivos

1.2.1 Objetivo Geral

Investigar as percepções e os fatores que influenciam a adoção de JavaScript e TypeScript no desenvolvimento front-end, analisando as diferenças entre desenvolvedores iniciantes e profissionais experientes.

1.2.2 Objetivos Específicos

1. Analisar comparativamente as características técnicas do JavaScript e do TypeScript, com foco nos sistemas de tipagem e seus impactos no desenvolvimento.
2. Identificar, com base na literatura e em dados, os benefícios e desafios associados à adoção do TypeScript em projetos de software.
3. Avaliar de que forma o nível de experiência dos desenvolvedores (iniciantes versus experientes) influencia a percepção e a preferência por uma ou outra tecnologia.

2 Fundamentação Teórica

A presente análise teórica explora os conceitos que fundamentam a coexistência entre JavaScript e TypeScript no desenvolvimento de aplicações web. O percurso argumentativo parte da tipagem dinâmica e da assincronicidade do JavaScript, apresenta o TypeScript como uma forma de tipagem gradual e discute os limites dessa proteção. Em seguida, examina contratos de bibliotecas, configuração do compilador, ferramentas de migração, dívida técnica, colaboração entre equipes e a mudança do perfil de falhas em ecossistemas modernos, posicionando a experiência do desenvolvedor como um fator relevante, mas não isolado, na escolha tecnológica.

2.1 JavaScript, tipagem dinâmica e desenvolvimento web

O desenvolvimento front-end foi profundamente moldado pelo papel central do *JavaScript*, linguagem criada em 1995 por Brendan Eich. Embora concebido inicialmente como um recurso de script para adicionar interatividade a páginas estáticas, sua evolução, marcada pela padronização como *ECMAScript*, o consolidou como a principal tecnologia para aplicações web, garantindo consistência entre diferentes ambientes de execução e tornando-se o padrão universal na camada de interação com o usuário (Mororó, 2024). Uma de suas características mais marcantes é a tipagem dinâmica, na qual os tipos de dados estão associados aos valores, e não às variáveis. Essa flexibilidade permite uma escrita de código ágil, tornando a linguagem atrativa para prototipagem e projetos menores, onde uma mesma variável pode mudar de tipo dinamicamente sem que o desenvolvedor precise de declarações prévias.

```
let resultado = 100; // 'resultado' é inferido como number
resultado = 'cem';   // Válido em JS. Agora 'resultado' é uma string.
```

Contudo, essa mesma liberdade introduz uma classe de desafios em sistemas complexos. Como aponta (Makimov, 2022), erros de tipo, como a passagem de dados em formato inesperado para uma função, podem só se manifestar em tempo de execução. Tal característica eleva o custo de manutenção, pois a depuração de erros em grandes bases de código torna-se mais árdua. A essa complexidade soma-se o modelo de concorrência do JavaScript, baseado em um *event loop*. Embora eficaz para impedir que operações demoradas bloqueiem a interface, a gestão do fluxo assíncrono permanece uma fonte de condições de corrida, falhas de tratamento e comportamentos difíceis de reproduzir. A introdução de *Promises* e, posteriormente, da sintaxe *async/await*, melhorou a legibilidade, mas não substituiu a necessidade de testes e validações em execução (Mororó, 2024; Tang; Alimadadi; Sumner, 2026).

Foi nesse cenário de um ecossistema poderoso, porém propenso a desafios de escalabilidade, que a Microsoft lançou o *TypeScript* em 2012. Definido como um superconjunto de JavaScript, sua principal inovação foi a introdução de um sistema de tipagem estática opcional. Essa abordagem significa que todo código JavaScript é, por definição, um código TypeScript válido, o que viabiliza uma migração gradual e estratégica. A proposta central do TypeScript é deslocar a verificação de tipos do tempo de execução para o tempo de compilação, proporcionando maior segurança e melhorando a legibilidade, visto que o código se torna, em parte, autodescritivo (George, 2020; Makimov, 2022). Um erro de atribuição de tipo, por exemplo, é capturado antes mesmo que o programa seja executado.

2.2 TypeScript como sistema de tipagem gradual

```
let meuNome: string = "Gustavo";  
// O compilador do TypeScript apontará um erro nesta linha:  
// Erro: O tipo 'number' não pode ser atribuído ao tipo 'string'.  
meuNome = 30;
```

A promessa de maior robustez do TypeScript deve, portanto, ser interpretada de forma condicional. Um estudo de (Richards; Nardelli; Vitek, 2017) estimou que a tipagem estática poderia prevenir cerca de 15% dos bugs públicos em projetos JavaScript. Por outro lado, a análise de (Bogner; Wagner, 2022) indicou que aplicações TypeScript podem apresentar melhor qualidade de código sem necessariamente possuir menos bugs. Esse resultado é compatível com a ideia de que a tipagem reduz determinadas falhas locais, mas não substitui a análise da lógica, dos requisitos e das integrações.

Essa distinção é reforçada por estudos sobre o próprio TypeScript. Wang, Fang e Wang (Wang; Fang; Wang, 2025) identificaram uma variedade de defeitos ligados a nomes, bindings, escopos, tipos, semântica, recursos da linguagem e comportamento do compilador. Em uma escala mais ampla, Tang, Alimadadi e Sumner (Tang; Alimadadi; Sumner, 2026) analisaram 633 relatos de bugs em 16 projetos e observaram a predominância de problemas de configuração e ferramentas, uso incorreto de APIs e tratamento assíncrono. Os resultados não autorizam concluir que o TypeScript seja menos confiável; indicam que a adoção desloca parte da atenção necessária para as camadas de integração e orquestração.

Para além da tipagem, o TypeScript oferece interfaces, tipos genéricos e mecanismos de composição que podem funcionar como documentação e apoio à refatoração. Entretanto, uma declaração de tipo só é útil quando representa o comportamento real do componente. Feldthaus e Møller (Feldthaus; Møller, 2014) verificaram a correção de interfaces TypeScript para bibliotecas JavaScript e demonstraram a importância de testar o contrato contra a implementação. De modo semelhante, Hoefflich, Findler e Serrano (Hoefflich; Findler; Serrano, 2022) desenvolve-

ram o Scotty para encontrar incompatibilidades no repositório DefinitelyTyped, mostrando que declarações incorretas podem produzir uma sensação indevida de segurança. A fronteira entre o tipo declarado e o comportamento efetivo da biblioteca é, portanto, um elemento central da confiabilidade.

Apesar dos benefícios, a adoção do TypeScript impõe uma curva de aprendizado e custos de configuração. Desenvolvedores precisam compreender novos conceitos, integrar compilador, bundler, testes e linters e decidir quanto rigor aplicar ao projeto. Em estudo com mais de 4.000 repositórios, Zandonotto et al. (Zandonotto *et al.*, 2026) encontraram associação entre opções como `noImplicitAny` e `noUnusedParameters` e menores índices de problemas estáticos, mas também observaram correlações contraintuitivas para algumas opções. Como o estudo é correlacional, seus resultados não constituem garantia causal; ainda assim, evidenciam que a simples presença de arquivos `.ts` não define o nível de proteção obtido.

A migração pode ser apoiada por ferramentas de inferência e geração automática. O dataset ManyTypes4TypeScript reúne projetos e anotações de tipos em escala para pesquisas de inferência (Jesse; Devanbu, 2022), enquanto Jesse, Devanbu e Sawant (Jesse; Devanbu; Sawant, 2023) investigaram a previsão de tipos definidos por usuários. Cristiani e Thiemann (Cristiani; Thiemann, 2021) estudaram a geração de arquivos de declaração a partir de JavaScript. Essas técnicas podem reduzir o custo inicial, mas não eliminam a revisão humana: uma inferência ou declaração incorreta pode propagar um contrato falso pelo sistema.

Como aponta (Shrestha *et al.*, 2022), o conhecimento prévio de JavaScript também pode interferir na adaptação ao novo paradigma. Ainda assim, a integração com frameworks e ferramentas modernas e os ganhos percebidos em projetos de grande porte ajudam a explicar a relevância do TypeScript (Stack Overflow, 2023). A literatura, portanto, não sustenta uma oposição simples entre uma linguagem melhor e outra pior. O JavaScript permanece vantajoso para aprendizagem, prototipagem e situações em que a agilidade inicial é prioritária; o TypeScript tende a oferecer maior apoio à manutenção e à coordenação em bases extensas, desde que acompanhado por contratos confiáveis, configuração coerente, testes e validações de runtime. Esse equilíbrio orienta a análise empírica deste trabalho.

2.3 Qualidade de software, manutenção e dívida técnica

A comparação entre JavaScript e TypeScript também pode ser interpretada pela perspectiva da dívida técnica. Alves et al. (Alves *et al.*, 2016) realizaram um mapeamento sistemático de 100 estudos e mostram que a dívida técnica reúne decisões e artefatos que oferecem benefícios imediatos, mas podem produzir custos futuros de manutenção, evolução e qualidade. A metáfora financeira é útil porque permite distinguir o benefício obtido no presente dos chamados juros: quanto mais tempo uma decisão inadequada permanece sem revisão, maior pode ser

o esforço necessário para compreendê-la, corrigi-la ou substituí-la.

No desenvolvimento web, a dívida pode surgir em diferentes camadas. No código, pode resultar de funções pouco documentadas, uso excessivo de valores genéricos ou ausência de contratos explícitos. Na arquitetura, pode aparecer quando a equipe adia a modularização ou acopla componentes para entregar uma funcionalidade rapidamente. No processo, pode ser consequência de testes insuficientes, configuração improvisada ou ausência de critérios de revisão. Nessa perspectiva, a escolha por JavaScript não constitui automaticamente uma dívida técnica, assim como a adoção de TypeScript não garante sua eliminação. O efeito depende do contexto, da escala do sistema e da capacidade da equipe de administrar as consequências da decisão.

Os resultados deste TCC podem ser lidos nessa chave. A média de 4,29 atribuída pelos iniciantes à rapidez de prototipagem em JavaScript representa um benefício de curto prazo percebido. Em contrapartida, a média de 4,44 atribuída pelos experientes à manutenibilidade do TypeScript sugere que, em bases submetidas a mudanças contínuas, contratos e ferramentas de refatoração podem compensar o investimento inicial. A diferença não significa que todos os projetos devam começar com a mesma configuração; significa que a decisão deve considerar o horizonte de evolução e o custo de adiar uma estruturação necessária.

2.3.1 Dívida técnica como decisão de gestão

A dívida técnica não é apenas um problema interno do código. Gomes et al. ([Gomes et al., 2023](#)) analisaram 108 discussões, 547 publicações e 882 comentários no Stack Exchange e identificaram 74 indicadores e 126 práticas de gerenciamento. O estudo constatou que profissionais de gerenciamento discutem com maior frequência dívidas de processo e de pessoas, enquanto trabalhos centrados em desenvolvedores tendem a enfatizar dívidas de código e design. Essa diferença é importante para a comparação entre linguagens: a adoção de TypeScript pode melhorar a expressividade do código, mas não resolve sozinha problemas de treinamento, comunicação, prioridades, revisão ou planejamento de migração.

Consequentemente, uma decisão tecnológica madura deve incluir critérios de governança. Entre eles estão a definição de níveis de rigor do compilador, a política para uso de `any`, a exigência de testes para contratos de APIs, a revisão das declarações de bibliotecas e o acompanhamento do custo de manutenção. Essa abordagem impede que a tipagem seja tratada como uma promessa abstrata e a transforma em parte de um processo verificável de qualidade.

2.4 Contratos externos, integração e arquitetura

O código front-end raramente funciona de forma isolada. Ele consome APIs, bibliotecas, frameworks, serviços de autenticação e componentes de infraestrutura. Em sistemas distri-

buídos, a confiabilidade depende da coordenação entre esses elementos. Santana et al. (Santana; Andrade *et al.*, 2021) discutem a disponibilidade de aplicações IoT baseadas em microsserviços reativos, destacando a necessidade de lidar com comunicação entre serviços, falhas e evolução de componentes. Embora o estudo não compare JavaScript e TypeScript, ele ajuda a situar a linguagem dentro de uma arquitetura maior, na qual o contrato de um módulo é apenas uma parte do comportamento observado.

No mesmo sentido, Santana, Alencar e Prazeres (Santana; Alencar; Prazeres, 2019) analisam microsserviços reativos em um cenário de computação em névoa. A contribuição para este trabalho é contextual: em arquiteturas distribuídas, disponibilidade, interoperabilidade e escalabilidade dependem de fronteiras bem definidas e de tratamento explícito de falhas. Assim, o ganho do TypeScript em um contrato local não substitui validações de mensagens, controle de versões, observabilidade e testes de integração.

Essa discussão complementa os estudos diretamente voltados ao ecossistema TypeScript. Feldthaus e Møller (Feldthaus; Møller, 2014) e Hoefflich, Findler e Serrano (Hoefflich; Findler; Serrano, 2022) demonstram que declarações de tipos podem divergir da implementação JavaScript. O contrato estático deve, portanto, ser tratado como hipótese que precisa ser confrontada com o comportamento efetivo, especialmente nas fronteiras com bibliotecas externas e serviços remotos.

2.5 Configuração, ferramentas e migração

A efetividade do TypeScript depende da configuração e do conjunto de ferramentas que cercam o compilador. A aplicação combina arquivos de configuração, resolução de módulos, bundlers, transpiladores, linters, formatadores, ambientes de teste e plataformas de execução. Uma alteração aparentemente local pode produzir efeitos em várias etapas do pipeline. Tang, Alimadadi e Sumner (Tang; Alimadadi; Sumner, 2026) encontraram, em 633 relatos de 16 projetos, predominância de falhas de configuração e ferramentas, uso inadequado de APIs e problemas assíncronos. O resultado reforça que o aumento da estrutura também cria novas superfícies de integração.

Zandonotto et al. (Zandonotto *et al.*, 2026) analisaram mais de 4.000 repositórios e relacionaram opções do `tsconfig.json` a métricas de bugs e *code smells*. Opções como `noImplicitAny` e `noUnusedParameters` apresentaram associação com menos problemas estáticos, enquanto outras tiveram correlações contraintuitivas. Como se trata de associação, não se pode afirmar que uma opção isolada cause melhora ou piora; projetos mais rigorosos podem ser justamente aqueles que já investem mais em qualidade. Ainda assim, o estudo sustenta que o arquivo de configuração é uma decisão de engenharia, não um detalhe operacional.

A migração gradual pode usar inferência automática, geração de declarações e bases de

exemplos reais. Jesse e Devanbu ([Jesse; Devanbu, 2022](#)) apresentam o ManyTypes4TypeScript, dataset com ampla diversidade de projetos e anotações, e Jesse, Devanbu e Sawant ([Jesse; Devanbu; Sawant, 2023](#)) investigam a previsão de tipos definidos por usuários. Cristiani e Thiemann ([Cristiani; Thiemann, 2021](#)) estudam a geração de arquivos de declaração a partir de JavaScript. Essas soluções reduzem o esforço mecânico, mas não substituem decisões semânticas: saber que um valor é uma string não informa necessariamente quais valores são válidos, em que momento podem ser usados ou qual regra de negócio representam.

2.6 Adoção, colaboração e maturidade da equipe

A escolha entre JavaScript e TypeScript é influenciada pela experiência acumulada, pela finalidade do projeto e pela cultura de trabalho. Shrestha et al. ([Shrestha et al., 2022](#)) discutem dificuldades de aprendizagem de novas linguagens e ajudam a explicar por que o conhecimento de JavaScript pode tanto facilitar a entrada no TypeScript quanto preservar hábitos que dificultam a compreensão de suas restrições. Iniciantes tendem a perceber o custo da modelagem imediatamente; profissionais experientes tendem a perceber com mais nitidez o retorno de refatorações seguras, navegação semântica e padronização.

Coutinho, Andrade e Santana ([Coutinho; Andrade; Santana, 2024](#)) analisam métodos ágeis em pesquisa colaborativa entre universidade e indústria. O estudo destaca adaptação de processos, alinhamento de objetivos, comunicação interdisciplinar e gestão das diferenças entre participantes. Essa perspectiva é adequada ao presente TCC porque a amostra reúne estudantes, iniciantes e profissionais com experiências distintas. A percepção sobre uma linguagem não deriva apenas de sua sintaxe: ela é formada por tarefas, prazos, responsabilidades, ferramentas e formas de colaboração.

Portanto, a adoção do TypeScript deve ser analisada como uma mudança sociotécnica. Ela envolve linguagem, compilador, bibliotecas, arquitetura, processo e aprendizagem. A literatura indica ganhos potenciais de qualidade e manutenção, mas também custos de configuração, contratos frágeis e deslocamento dos riscos para o toolchain. Esse quadro fundamenta a comparação empírica realizada neste trabalho e impede que os resultados sejam reduzidos a uma simples preferência entre tecnologias.

3 Metodologia

Para atingir os objetivos propostos neste trabalho, notadamente a investigação das percepções e dos fatores que influenciam a adoção de JavaScript e TypeScript entre diferentes perfis de desenvolvedores, optou-se por uma abordagem de métodos mistos, com predominância quantitativa e complemento qualitativo. O estudo é caracterizado como uma pesquisa aplicada, de natureza descritiva e comparativa, realizada por meio de levantamento (*survey*). O desenho quantitativo permite sintetizar respostas padronizadas, enquanto a análise qualitativa dos relatos abertos ajuda a explicar por que determinadas percepções aparecem nos dados. Essa combinação é coerente com o problema, pois a preferência tecnológica possui uma dimensão mensurável e outra relacionada às experiências de trabalho, manutenção e colaboração.

3.1 Problema de pesquisa e questões investigativas

O problema de pesquisa pode ser formulado da seguinte maneira: como desenvolvedores com diferentes níveis de experiência percebem os benefícios, custos e riscos associados ao uso de JavaScript e TypeScript no desenvolvimento front-end? A pergunta não pressupõe que uma tecnologia seja universalmente superior. Ela considera que prototipagem, aprendizagem, manutenção, refatoração, integração e governança podem produzir avaliações diferentes.

Com base nesse problema, o levantamento foi orientado pelas seguintes questões investigativas:

1. Como iniciantes e profissionais experientes avaliam produtividade, facilidade de aprendizagem e rapidez de prototipagem?
2. Como os grupos avaliam detecção precoce de erros, legibilidade, manutenibilidade e padronização?
3. Que dificuldades são percebidas na configuração, migração e integração com bibliotecas externas?
4. Em que medida a experiência em projetos de maior escala influencia a preferência pelo TypeScript?
5. Que benefícios e limitações aparecem nos relatos abertos sobre uso real das duas tecnologias?

Essas questões dialogam com a literatura sobre dívida técnica. Uma decisão que reduz o esforço inicial pode produzir custos posteriores de manutenção, enquanto uma configuração

mais rigorosa pode exigir investimento antecipado para reduzir problemas futuros (Alves *et al.*, 2016). Também dialogam com estudos que mostram que a percepção de gestores e desenvolvedores pode enfatizar dimensões diferentes da dívida técnica (Gomes *et al.*, 2023).

3.2 Instrumento de coleta

O instrumento de coleta de dados consistiu em dois questionários distintos, elaborados e distribuídos por meio da plataforma Google Forms. A decisão de criar dois formulários separados foi estratégica, visando adequar a linguagem e o foco das perguntas ao nível de conhecimento e ao contexto de cada público-alvo, sem alterar o núcleo das dimensões comparadas. Ambos os questionários foram estruturados com perguntas fechadas, utilizando escalas de Likert de cinco pontos, perguntas de múltipla escolha e uma questão aberta sobre experiências práticas.

As afirmações foram organizadas em quatro blocos analíticos. O Bloco A avaliou produtividade, agilidade, curva de aprendizagem, prototipagem e detecção precoce de erros. O Bloco B avaliou legibilidade, manutenibilidade, origem de bugs, verbosidade e padronização. O Bloco C abordou compatibilidade com bibliotecas, ferramentas, frameworks, migração e projetos legados. O Bloco D examinou confiança, preferência para novos projetos, percepção de futuro e adequação a equipes grandes. A pergunta aberta foi incluída para identificar situações não previstas nas alternativas fechadas.

Essa estrutura procura operacionalizar os principais construtos do referencial. A detecção precoce representa o benefício esperado da verificação estática; manutenibilidade e refatoração se relacionam ao custo de evolução; migração e configuração representam o investimento inicial; e integração com bibliotecas representa a confiabilidade dos contratos externos. A inclusão de processo, equipe e colaboração é importante porque a literatura mostra que a dívida técnica também envolve pessoas e decisões organizacionais, não apenas o código (Gomes *et al.*, 2023; Coutinho; Andrade; Santana, 2024).

3.3 População, amostragem e grupos de análise

O primeiro grupo-alvo foi composto por **desenvolvedores profissionais experientes**, atuantes no mercado de desenvolvimento de software. A amostra foi selecionada por conveniência, utilizando redes de contatos profissionais, como o LinkedIn, e comunidades online de desenvolvedores. Para manter a coerência com a segmentação apresentada nos resultados, foram considerados experientes os participantes com quatro ou mais anos de experiência em desenvolvimento front-end. O grupo é relevante para compreender percepções formadas em projetos

comerciais, de longa duração e com equipes colaborativas, nos quais manutenibilidade, escalabilidade e governança assumem maior importância.

O segundo grupo foi formado por **desenvolvedores iniciantes e intermediários**, majoritariamente estudantes do curso de Bacharelado em Sistemas de Informação do Instituto Federal da Bahia (IFBA), Campus Feira de Santana, e de outras instituições. Para a comparação principal, foram agrupados participantes com até três anos de experiência. A seleção também foi por conveniência, por meio de convites em grupos de estudo e canais acadêmicos. Esse grupo representa participantes ainda próximos do processo de aprendizagem e, por isso, é especialmente adequado para investigar curva de aprendizado, simplicidade inicial e percepção do custo de configuração (Shrestha *et al.*, 2022).

A amostra final foi composta por 23 respondentes: 14 no grupo iniciante/intermediário e 9 no grupo experiente. Doze participantes declararam experiência em projetos de grande escala e 11 relataram atuação predominantemente em projetos individuais, acadêmicos ou de menor porte. Essa variável foi utilizada como dimensão complementar, pois a escala do sistema altera a relevância de contratos, refatorações e coordenação entre pessoas.

3.4 Procedimentos de coleta e tratamento dos dados

Após a divulgação dos formulários, as respostas foram exportadas para planilhas eletrônicas e submetidas a conferência. Foram verificadas duplicidades, respostas incompletas e a correspondência entre as respostas e o grupo de experiência informado. Os dados foram então codificados numericamente, preservando a direção da escala: valores maiores representam maior concordância com a afirmação apresentada. As respostas abertas foram mantidas em texto para a etapa qualitativa.

Não se pretendeu produzir uma estimativa probabilística da população de desenvolvedores. A amostragem por conveniência permite explorar tendências e contrastes dentro do grupo alcançado, mas exige cautela na generalização. Por esse motivo, os resultados são interpretados como evidências descritivas sobre as percepções dos respondentes, e não como prova de que todos os iniciantes ou experientes se comportam da mesma maneira.

3.5 Plano de análise quantitativa

A análise quantitativa utilizou estatística descritiva, incluindo frequências absolutas e relativas, médias, medianas e desvios-padrão. Para cada item, foram calculadas as respostas da amostra total e dos dois grupos principais. A diferença perceptual entre experientes e iniciantes

foi expressa por:

$$\Delta = \bar{x}_{experientes} - \bar{x}_{iniciantes}.$$

Valores positivos indicam maior concordância entre experientes; valores negativos indicam maior concordância entre iniciantes. A mediana foi apresentada junto à média porque a escala de Likert é ordinal e pode concentrar respostas em determinados pontos. O desvio-padrão foi usado como indicador descritivo de dispersão, permitindo distinguir consenso de respostas mais heterogêneas.

As comparações foram organizadas por dimensão, e não apenas por item isolado. Essa decisão reduz o risco de interpretar uma resposta específica fora do contexto e permite relacionar os dados aos construtos teóricos: produtividade, qualidade, integração e preferência. As diferenças foram discutidas em conjunto com o tamanho dos grupos, a experiência em projetos grandes e os relatos qualitativos. Não foram feitas afirmações causais a partir das médias, pois o desenho transversal e a amostragem não permitem concluir que a experiência provoque determinada preferência.

3.6 Plano de análise qualitativa

As respostas abertas foram examinadas por análise temática categorial. Inicialmente, foram identificadas unidades de sentido relacionadas a benefícios, custos, contexto de uso, integração, manutenção e execução. Em seguida, os relatos foram agrupados em quatro eixos: blindagem de contratos e APIs; refatoração em larga escala; complexidade inicial e fricção com o ecossistema; e fronteira entre compilação e execução.

Os eixos não foram tratados como categorias mutuamente exclusivas de defeitos. Um mesmo relato poderia mencionar tipagem, biblioteca externa e validação de runtime. A categorização teve como finalidade aproximar a experiência dos participantes dos conceitos encontrados na literatura, especialmente dívida técnica, contratos quebrados, toolchain e colaboração. A interpretação buscou preservar o sentido dos depoimentos e evitar transformar opiniões individuais em generalizações sobre toda a comunidade.

3.7 Validade, limitações e aspectos éticos

Há ameaças à validade interna, externa e de construção. A validade interna é limitada porque as respostas são autorreferidas e podem refletir preferência pessoal, familiaridade com ferramentas ou experiências específicas. A validade externa é limitada pela amostragem por conveniência e pelo número reduzido de participantes. A validade de construção exige cau-

tela porque termos como produtividade, segurança e manutenibilidade são complexos e foram medidos por percepção, não por métricas de repositórios ou experimentos controlados.

Para reduzir esses riscos, foram utilizadas perguntas organizadas por dimensões, comparação entre grupos, médias acompanhadas de medianas e desvios-padrão e triangulação com respostas abertas. A literatura também foi usada para interpretar os resultados sem reduzir qualidade a uma única variável. Estudos de mineração de repositórios, como os que investigam opções do compilador e categorias de bugs, servem como referência externa, mas não substituem a análise do survey, pois empregam unidades de análise e objetivos diferentes.

No plano ético, a participação foi voluntária e os dados foram analisados de forma agregada. Os relatos apresentados no capítulo de resultados são identificados por códigos de respondentes, sem exposição de nomes ou informações pessoais desnecessárias. O uso de depoimentos teve finalidade acadêmica e foi restrito ao conteúdo relacionado ao problema de pesquisa.

3.8 Síntese metodológica

Em síntese, o método combina levantamento quantitativo, comparação entre níveis de experiência e interpretação qualitativa de relatos. Essa estratégia é adequada ao objetivo de investigar não apenas qual linguagem recebe maior preferência, mas quais razões sustentam essa preferência. A abordagem também permite compreender a escolha como decisão sociotécnica: a linguagem interage com configuração, contratos, arquitetura, processo, aprendizagem e colaboração. Dessa forma, os resultados podem ser confrontados com a literatura de qualidade e dívida técnica sem extrapolar o alcance da amostra.

4 Análise e Discussão dos Resultados

Este capítulo apresenta a consolidação, análise e discussão aprofundada dos dados empíricos obtidos a partir da aplicação dos questionários estruturados a desenvolvedores de software front-end. O objetivo central é confrontar as hipóteses teóricas estabelecidas na literatura com as percepções reais da prática profissional, identificando como fatores como tempo de atuação na área e vivência em projetos de grande escala moldam a avaliação de JavaScript e TypeScript.

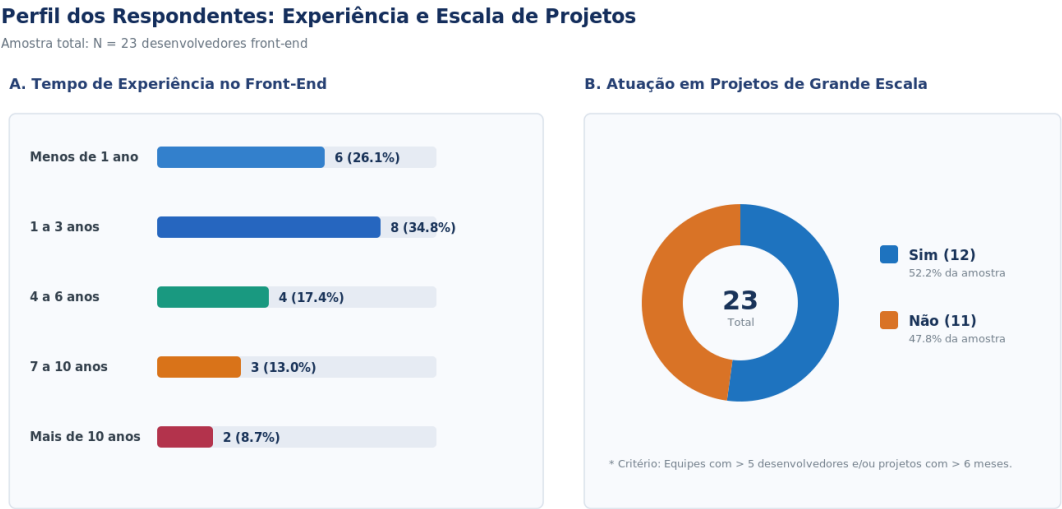
A análise está estruturada em seis seções: inicialmente, caracteriza-se o perfil demográfico da amostra; em seguida, examinam-se as dimensões de produtividade e agilidade de desenvolvimento (Bloco A); na sequência, analisam-se os aspectos de qualidade, manutenibilidade e prevenção de falhas (Bloco B); posteriormente, avalia-se o ecossistema, o suporte de ferramentas e os desafios de migração (Bloco C); discutem-se então as preferências pessoais, o nível de confiança e as perspectivas futuras (Bloco D); por fim, realiza-se uma categorização temática dos relatos qualitativos abertos e uma síntese de confronto com os referenciais teóricos.

4.1 Caracterização Demográfica dos Participantes

A pesquisa contou com a participação de um total de $N = 23$ desenvolvedores. A segmentação dos respondentes seguiu os parâmetros metodológicos delineados no Capítulo 3, permitindo agrupar a amostra de acordo com o tempo de experiência no desenvolvimento front-end e a vivência em projetos de grande escala (definidos como iniciativas com mais de 5 desenvolvedores e/ou duração superior a 6 meses).

A distribuição da amostra quanto ao tempo de experiência e participação em projetos corporativos de grande porte está ilustrada na Figura 1.

Figura 1 – Perfil dos respondentes por tempo de experiência e participação em grandes projetos



Fonte: Dados da pesquisa aplicada pelo autor (2025).

Fonte: Dados da pesquisa aplicada pelo autor (2025).

Conforme evidencia a Figura 1(A), os respondentes distribuem-se em diferentes estágios de maturidade na carreira:

- **Menos de 1 ano:** 6 participantes (26,1%);
- **1 a 3 anos:** 8 participantes (34,8%);
- **4 a 6 anos:** 4 participantes (17,4%);
- **7 a 10 anos:** 3 participantes (13,0%);
- **Mais de 10 anos:** 2 participantes (8,7%).

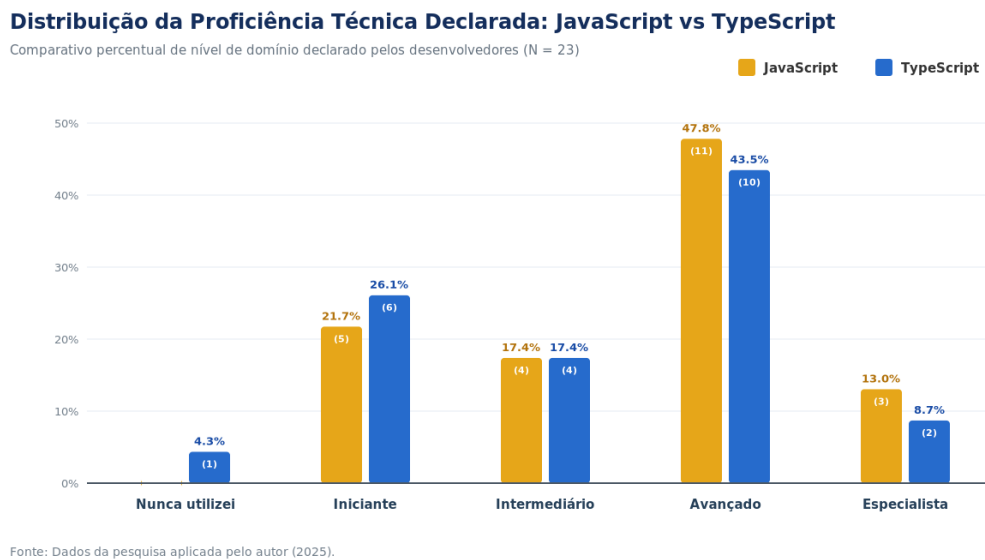
Para fins de análise comparativa neste estudo, foram estabelecidos dois grupos principais:

1. **Grupo de Desenvolvedores Iniciantes e Intermediários** ($N = 14$, **60,9%**): composto por profissionais e estudantes em fase de formação com até 3 anos de experiência em desenvolvimento front-end.
2. **Grupo de Profissionais Experientes** ($N = 9$, **39,1%**): formado por desenvolvedores com 4 ou mais anos de experiência comprovada no mercado de tecnologia, incluindo seniores e especialistas técnicos.

Adicionalmente, a Figura 1(B) demonstra que 12 participantes (52,2%) possuem experiência sólida em projetos de grande escala, enquanto 11 participantes (47,8%) atuaram predominantemente em projetos individuais, de menor porte ou em contexto estritamente acadêmico. Esse equilíbrio amostral é particularmente valioso para avaliar se as dores e os benefícios apreendidos pela adoção do TypeScript dependem da escala organizacional do software.

A Figura 2 apresenta a comparação entre os níveis declarados de proficiência técnica em JavaScript e em TypeScript.

Figura 2 – Comparativo de proficiência técnica autoavaliada em JavaScript e TypeScript



Fonte: Dados da pesquisa aplicada pelo autor (2025).

Os dados revelam que o domínio de JavaScript é consolidado na totalidade da amostra, com 60,8% dos participantes posicionando-se nos níveis *Avançado* (47,8%) ou *Especialista* (13,0%), e nenhum respondente declarando desconhecimento da linguagem. Em relação ao TypeScript, observa-se uma distribuição similar no topo da pirâmide (52,2% entre avançados e especialistas), porém com uma fração mais expressiva de iniciantes (26,1%) e um participante (4,3%) que declarou nunca ter utilizado a ferramenta. Esse panorama reflete com precisão a dinâmica descrita por (Mororó, 2024), em que o JavaScript atua como fundação prévia obrigatória para a assimilação subsequente do TypeScript.

4.2 Produtividade e Desenvolvimento Ágil (Bloco A)

O Bloco A do questionário investigou a percepção dos participantes acerca da velocidade de desenvolvimento, facilidade de prototipagem, curva de aprendizado e detecção de

falhas. As respostas foram mensuradas por meio de uma escala Likert de 5 pontos (variando de 1 = Discordo totalmente a 5 = Concordo totalmente).

A Tabela 1 sumariza as métricas estatísticas descritivas (médias, medianas e desvios-padrão) para o conjunto geral da amostra e a desagregação comparativa entre desenvolvedores iniciantes e experientes.

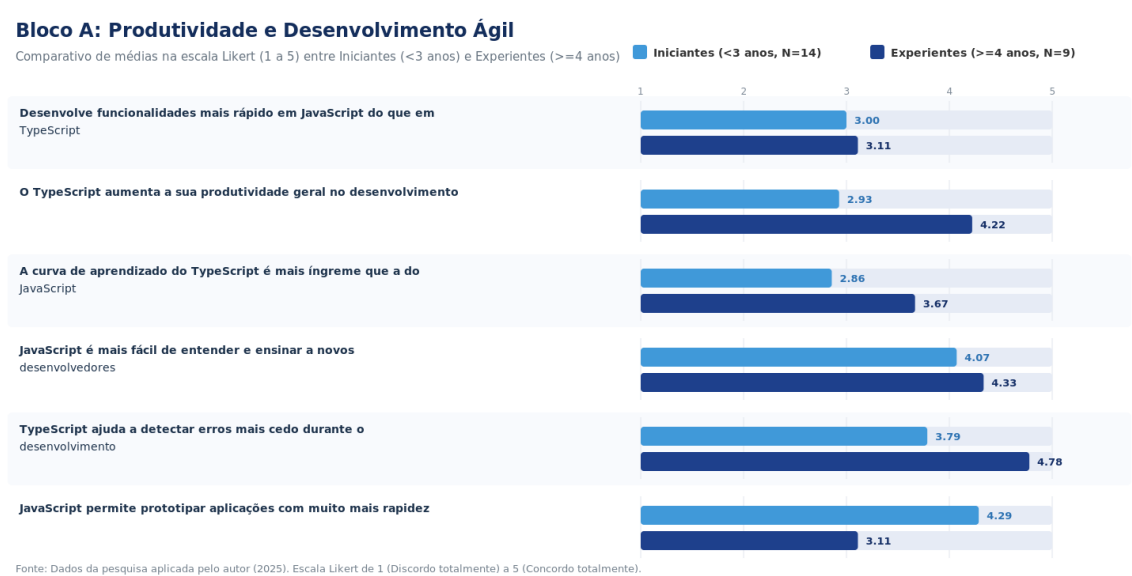
Tabela 1 – Estatísticas descritivas do Bloco A: Produtividade e Desenvolvimento

Item Avaliado	Média	Med.	DP	Inic.	Exp.	Δ (E-I)
A1. Desenvolve mais rápido em JS do que em TS	3,04	3,0	1,22	3,00	3,11	+0,11
A2. O TS aumenta a produtividade no desenvolvimento	3,43	4,0	1,41	2,93	4,22	+1,29
A3. A curva de aprendizado do TS é mais íngreme	3,17	3,0	1,07	2,86	3,67	+0,81
A4. JS é mais fácil de entender e ensinar a novos	4,17	4,0	0,89	4,07	4,33	+0,26
A5. O TS ajuda a detectar erros mais cedo	4,17	5,0	1,19	3,79	4,78	+0,99
A6. O JS permite prototipar aplicações mais rápido	3,83	4,0	1,23	4,29	3,11	-1,18

Fonte: Dados da pesquisa aplicada pelo autor (2025).

A representação gráfica comparativa dessas percepções é ilustrada na Figura 3.

Figura 3 – Comparativo de médias no Bloco A entre Iniciantes e Experientes



Fonte: Dados da pesquisa aplicada pelo autor (2025).

A análise dos resultados do Bloco A evidencia conclusões fundamentais sobre o impacto da maturidade profissional na percepção de produtividade:

1. **Diferença de percepção de produtividade** ($\Delta = +1, 29$): Enquanto os desenvolvedores iniciantes mantêm uma postura neutra ou reticente quanto aos ganhos de produtividade proporcionados pelo TypeScript (média de 2, 93), os profissionais experientes manifestam maior concordância com essa afirmação (média de 4, 22). Na amostra estudada, essa percepção pode estar relacionada ao uso de autocompletar, refatoração e detecção de inconsistências, mas o desenho da pesquisa não mede produtividade objetiva.
2. **Prototipagem rápida e flexibilidade** ($\Delta = -1, 18$): O grupo de iniciantes concorda mais fortemente que o JavaScript puro possibilita criar protótipos rapidamente (média de 4, 29), enquanto o grupo experiente apresenta média de 3, 11. Essa diferença é compatível com a percepção de que a configuração inicial do TypeScript pode ser mais custosa, mas não permite concluir que a linguagem seja objetivamente mais lenta em todos os contextos.
3. **Detecção precoce de erros** ($\Delta = +0, 99$): Ambos os grupos reconhecem o papel do TypeScript na identificação de defeitos antes do envio para produção, com média de 4, 78 e mediana 5, 0 entre experientes. O resultado é compatível com os achados de (Richards; Nardelli; Vitek, 2017) e (Makimov, 2022), mas representa percepção dos respondentes, não uma medição direta de defeitos evitados.
4. **Facilidade Pedagógica**: Tanto iniciantes (4, 07) quanto experientes (4, 33) concordam amplamente que o JavaScript é uma linguagem didaticamente mais acessível para a introdução de novos programadores à web, reflexo direto de sua sintaxe não vinculada à tipagem estrita prévia.

—

4.3 Qualidade do Código, Manutenibilidade e Prevenção de Falhas (Bloco B)

O Bloco B concentrou-se na análise das características intrínsecas da base de código: legibilidade, facilidade de manutenção em longo prazo, propensão à geração de defeitos, verbosidade e padronização arquitetural em equipes multidisciplinares.

Os dados estatísticos consolidados do Bloco B estão descritos na Tabela 2.

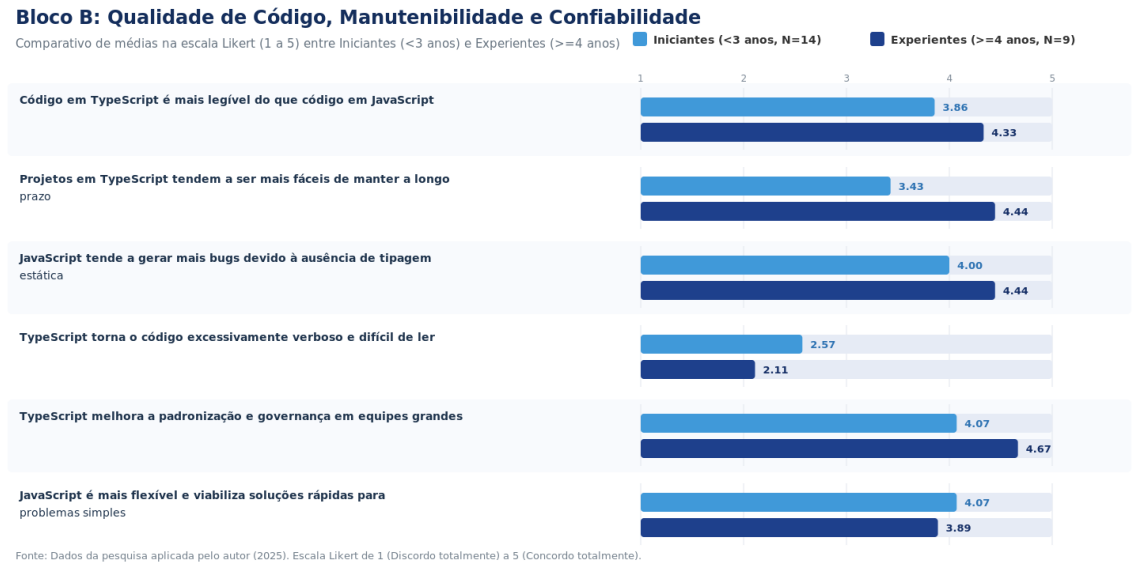
Tabela 2 – Estatísticas descritivas do Bloco B: Qualidade e Manutenção do Código

Item Avaliado	Média	Med.	DP	Inic.	Exp.	Δ (E-I)
B1. Código em TS é mais legível que em JS	4,04	5,0	1,33	3,86	4,33	+0,47
B2. Projetos em TS são mais fáceis de manter a longo prazo	3,83	4,0	1,34	3,43	4,44	+1,01
B3. JS gera mais bugs por ausência de tipagem	4,17	5,0	1,15	4,00	4,44	+0,44
B4. O TS torna o código excessivamente verboso e difícil	2,39	2,0	1,16	2,57	2,11	-0,46
B5. TS melhora a padronização em equipes grandes	4,30	5,0	0,88	4,07	4,67	+0,60
B6. JS facilita soluções rápidas para problemas simples	4,00	4,0	1,04	4,07	3,89	-0,18

Fonte: Dados da pesquisa aplicada pelo autor (2025).

A Figura 4 ilustra graficamente o comportamento comparativo das variáveis avaliadas no Bloco B.

Figura 4 – Comparativo de médias no Bloco B entre Iniciantes e Experientes



Fonte: Dados da pesquisa aplicada pelo autor (2025).

Os resultados do Bloco B reforçam aspectos cruciais da literatura de Engenharia de Software:

- 1. Manutenibilidade em longo prazo ($\Delta = +1,01$):** A assertiva de que bases de código estruturadas em TypeScript são mais sustentáveis recebeu avaliação destacada dos profissionais experientes (média de 4,44). Ao segmentar a amostra por participação em grandes projetos, a média foi de 4,42 entre participantes com essa experiência, contra 3,18 entre os demais. Esses dados sugerem uma associação entre vivência em sistemas maiores e

valorização da manutenibilidade, mas não permitem estimar retorno sobre investimento nem afirmar crescimento causal.

2. **Origem de Bugs por Ausência de Tipagem Estática:** A premissa de que a tipagem dinâmica do JavaScript é um vetor prolífico de defeitos em tempo de execução obteve alta concordância em ambos os grupos (4,00 entre iniciantes e 4,44 entre experientes; média geral de 4,17). Tal constatação alinha-se às observações de (Bogner; Wagner, 2022) e (George, 2020), ratificando que a manipulação de estruturas de dados sem contratos explícitos gera vulnerabilidades funcionais recorrentes.
3. **Desmistificação da Sobrecarga de Verbosidade:** A hipótese de que o TypeScript tornaria o código excessivamente prolixo ou ilegível foi amplamente rejeitada pelos participantes (média global de 2,39; 2,11 entre experientes e 2,00 entre os atuantes em grandes projetos). Em contrapartida, a legibilidade do código em TypeScript foi avaliada positivamente (4,04 geral e 4,33 para experientes), indicando que os desenvolvedores encaram as anotações de tipo não como ruído, mas como documentação viva acoplada ao código-fonte.
4. **Governança em Equipes Grandes:** O item que associa o TypeScript ao aprimoramento da padronização e consistência em times numerosos obteve a maior média de todo o bloco (4,30 geral e 4,67 entre experientes, com mediana 5,0). Esse dado corrobora a tese de que os tipos funcionam como um contrato formal de comunicação interequipes.

—

4.4 Compatibilidade, Ferramentas e Desafios de Migração (Bloco C)

O Bloco C investigou a relação do TypeScript com o ferramental moderno de desenvolvimento (IDEs, linters, formatadores), o ecossistema de frameworks de ponta (React, Angular, Vue) e os custos e dificuldades de integrar ou migrar sistemas concebidos originalmente em JavaScript.

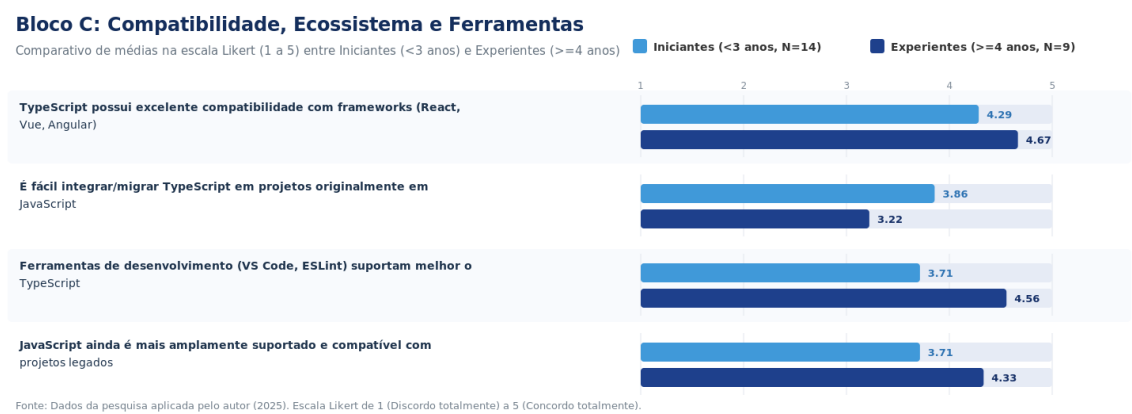
A Tabela 3 e a Figura 5 sintetizam as respostas obtidas.

Tabela 3 – Estatísticas descritivas do Bloco C: Compatibilidade e Ferramentas

Item Avaliado	Média	Med.	DP	Inic.	Exp.	Δ (E-I)
C1. TS possui excelente suporte em bibliotecas modernas	4,43	5,0	0,66	4,29	4,67	+0,38
C2. É fácil migrar/integrar TS em projetos legados JS	3,61	4,0	1,12	3,86	3,22	-0,64
C3. Ferramentas (IDEs/Linters) suportam melhor o TS	4,04	4,0	0,93	3,71	4,56	+0,85
C4. JS ainda é mais amplamente suportado	3,96	4,0	1,07	3,71	4,33	+0,62

Fonte: Dados da pesquisa aplicada pelo autor (2025).

Figura 5 – Comparativo de médias no Bloco C entre Iniciantes e Experientes



Fonte: Dados da pesquisa aplicada pelo autor (2025).

Dentre os principais achados do Bloco C, destacam-se:

- 1. Simbiose com o ecossistema e o ferramental ($\Delta = +0,85$):** O suporte proporcionado por ambientes de desenvolvimento integrados e extensões de análise estática foi avaliado favoravelmente pelos profissionais experientes (média de 4,56). Na percepção dos respondentes, esses recursos apoiam refatoração, navegação semântica e autocompletar contextual.
- 2. Percepções sobre a migração de projetos legados ($\Delta = -0,64$):** Os desenvolvedores iniciantes avaliaram o processo de migração de JavaScript para TypeScript como relativamente fácil (média de 3,86), enquanto os profissionais experientes apresentaram média de 3,22. A diferença é compatível com a hipótese de que a vivência em bases extensas torna mais visíveis os custos de tipar pacotes de terceiros, gerenciar configurações e refatorar padrões dinâmicos, mas não permite atribuir uma causa única às respostas.
- 3. Compatibilidade com frameworks:** A compatibilidade do TypeScript com frameworks front-end recebeu média de 4,43 e desvio-padrão de 0,66. O resultado expressa avaliação

favorável dos participantes, mas não permite afirmar, sozinho, que toda a indústria tenha estabelecido um padrão único.

4.5 Preferências Pessoais, Confiança e Perspectivas Futuras (Bloco D)

O Bloco D aferiu o vínculo atitudinal e emocional dos desenvolvedores em relação às duas linguagens, mensurando aspectos como nível de confiança psicológica ao codificar, preferência para novos projetos (*greenfield*) e a projeção de relevância futura no mercado.

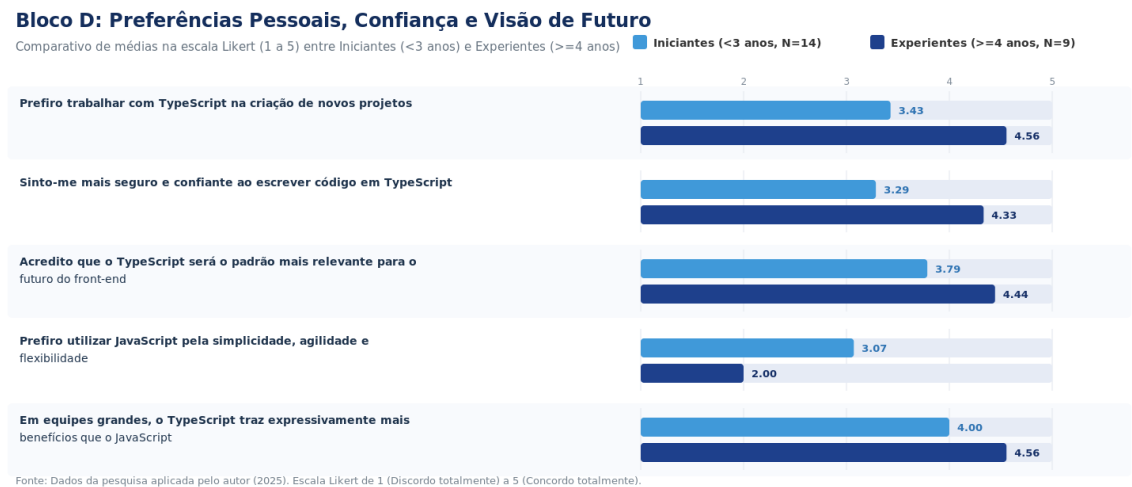
A Tabela 4 e a Figura 6 detalham as médias apuradas.

Tabela 4 – Estatísticas descritivas do Bloco D: Preferências e Perspectivas Pessoais

Item Avaliado	Média	Med.	DP	Inic.	Exp.	Δ (E-I)
D1. Prefiro trabalhar com TS em novos projetos	3,87	4,0	1,36	3,43	4,56	+1,13
D2. Sinto-me mais seguro e confiante com TS	3,70	4,0	1,33	3,29	4,33	+1,04
D3. Acredito que o TS será o futuro do front-end	4,04	4,0	1,11	3,79	4,44	+0,65
D4. Prefiro JS pela simplicidade e flexibilidade	2,65	3,0	1,50	3,07	2,00	-1,07
D5. Em equipes grandes, TS traz mais benefícios	4,22	5,0	1,00	4,00	4,56	+0,56

Fonte: Dados da pesquisa aplicada pelo autor (2025).

Figura 6 – Comparativo de médias no Bloco D entre Iniciantes e Experientes



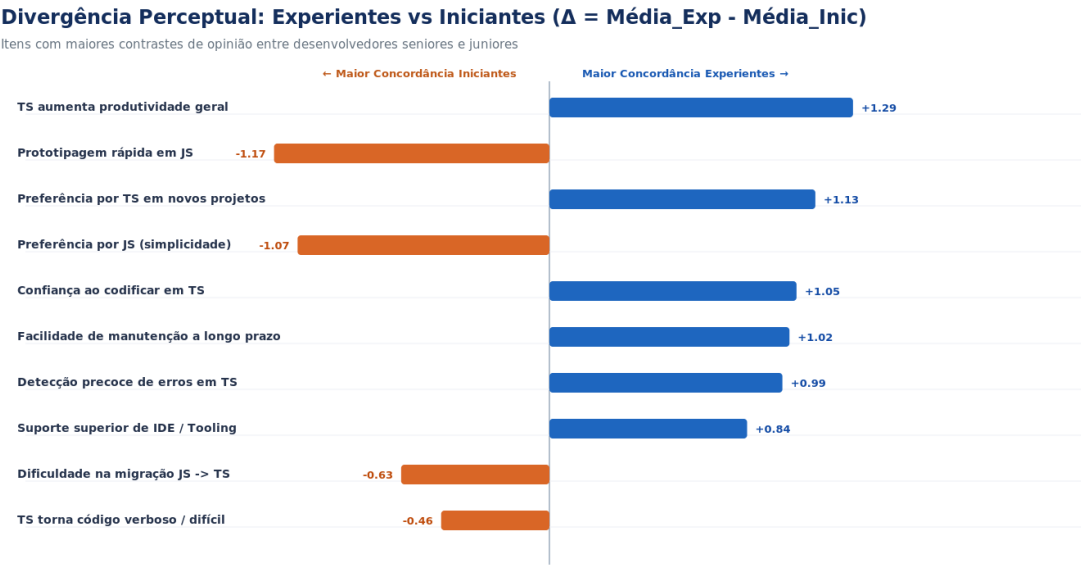
Fonte: Dados da pesquisa aplicada pelo autor (2025).

A dimensão atitudinal traz revelações expressivas:

1. **Confiança técnica percebida** ($\Delta = +1,04$): Os desenvolvedores experientes expressam maior confiança ao codificar em TypeScript (4,33 contra 3,29 dos iniciantes). Essa percepção pode estar relacionada ao apoio da validação estática, mas não demonstra, por si só, redução da carga cognitiva ou de erros reais.
2. **Preferência para novos projetos** ($\Delta = +1,13$): Diante da oportunidade de iniciar um projeto a partir do zero, os profissionais experientes apresentam maior preferência pelo TypeScript (4,56). A preferência pelo JavaScript devido à simplicidade foi menor nesse grupo (2,00) do que entre iniciantes (3,07), diferença que deve ser interpretada como percepção da amostra.
3. **Projeção de relevância futura**: Os participantes apresentaram média de 4,04 para a expectativa de expansão da relevância do TypeScript. Esse resultado descreve uma expectativa da amostra, não uma previsão objetiva do mercado.

A Figura 7 sintetiza as principais divergências perceptuais entre os dois grupos, ordenadas pela magnitude do delta ($\Delta = \text{Média}_{\text{Experientes}} - \text{Média}_{\text{Iniciantes}}$).

Figura 7 – Divergência perceptual consolidada entre Experientes e Iniciantes (Δ de Médias)



Fonte: Dados da pesquisa aplicada pelo autor (2025). Valores representam a diferença absoluta de médias entre os grupos.

Fonte: Dados da pesquisa aplicada pelo autor (2025).

4.6 Análise Qualitativa dos Relatos Práticos da Indústria

A questão aberta final do formulário solicitou aos participantes que relatassem, de forma contextualizada, experiências práticas onde o uso de TypeScript trouxe vantagens tangíveis ou desvantagens operacionais em relação ao JavaScript puro.

A análise temático-categorial desses depoimentos permitiu consolidar quatro eixos fundamentais que explicam a dinâmica real de adoção na indústria de software:

4.6.1 Eixo 1: Blindagem de Contratos de Dados e Integração de APIs

O benefício mais recorrente citado pelos profissionais foi o estabelecimento de interfaces e contratos estritos na comunicação entre camadas da aplicação e consumo de serviços REST/GraphQL.

Um desenvolvedor com mais de 10 anos de experiência (Respondente 13) relatou:

“Em projetos Node.js de médio e grande porte, o TypeScript trouxe ganhos significativos de qualidade ao permitir tipagem estática, melhor documentação do código e detecção precoce de erros durante o desenvolvimento. Em especial, em integrações entre múltiplos serviços e APIs, a tipagem ajudou a reduzir falhas de contrato entre componentes e tornou refatorações mais seguras.” (Respondente 13, Mais de 10 anos de experiência).

De forma complementar, o Respondente 1 destacou a aplicação em fluxos com regras de negócios críticas:

“Em projetos como o meu livrinho mágico e o portal da motopel o typescript ajudou porque deixou os dados mais seguros por exemplo: ao lidar com usuários, páginas do livro imagens e respostas da api ficaram mais fáceis de evitar erros, que no javascript só apareceriam com o sistema rodando.” (Respondente 1, 1 a 3 anos de experiência).

4.6.2 Eixo 2: Refatoração em Larga Escala e Propagação de Modificações

A segurança durante processos de refatoração constitui uma das maiores vantagens operacionais do TypeScript. A existência de tipos algébricos e uniões discriminadas permite que o compilador sinalize automaticamente todos os trechos do sistema que necessitam de adaptação após uma alteração de esquema.

O Respondente 5 (7 a 10 anos de experiência) ilustrou esse mecanismo de forma precisa:

“A atualização de um union type no typescript foi suficiente para gerar erros em todos os pontos que precisavam de atualização no código. Com javascript, seria necessária uma busca manual dos pontos de atualização, já que os erros não seriam manifestados.” (Respondente 5, Especialista em JS/TS).

Essa garantia mecânica elimina o risco de regressões silenciosas decorrentes de alterações em modelos de dados centrais, aspecto que frequentemente inviabiliza refatorações ambiciosas em projetos legados em JavaScript.

4.6.3 Eixo 3: Complexidade Inicial e Fricção com o Ecossistema Externo

Em contrapartida às vantagens de longo prazo, os participantes foram unânimes em apontar os custos de partida associados ao TypeScript: aumento do tempo de configuração de ambiente (*toolchain*), lentidão inicial para modelar interfaces complexas e atrito com bibliotecas que não disponibilizam definições de tipos adequadas.

O Respondente 12 pontuou essa relação de *trade-off*:

“Em um projeto utilizando Vue.js com Nuxt 3 e TypeScript, trabalhei na implementação de formulários complexos [...]. Houve desvantagens no aumento da complexidade inicial, principalmente ao lidar com tipagens mais avançadas e integração com bibliotecas externas que nem sempre possuem tipagem bem definida, exigindo ajustes manuais ou uso pontual de any. Mesmo assim, o ganho em segurança e escalabilidade compensou o custo inicial.” (Respondente 12, 4 a 6 anos de experiência).

Esse relato corrobora o achado quantitativo de que desenvolvedores seniores reconhecem a dificuldade de migração e configuração inicial, embora julguem que os benefícios posteriores em manutenibilidade compensem amplamente o custo despendido.

4.6.4 Eixo 4: A Fronteira entre Compilação e Execução (Runtime vs Compile-time)

Um aspecto técnico de extrema relevância, apontado com lucidez pelo Respondente 6, diz respeito à natureza da transpilação do TypeScript e seus limites de proteção:

“TypeScript por apresentar tipagem deixa o código muito mais legível e organizado, além de prevenir erros [...]. Porém o TS ele só vai validar e ver erros durante o desenvolvimento, em runtime ele ainda vai aceitar uma String em uma variável declarada como Number, porque por baixo ele vai rodar em JS. então você tem que

ficar atento para validar e tratar os dados que está mexendo.” (Respondente 6, 1 a 3 anos de experiência).

Essa observação adverte para a falsa sensação de segurança que o TypeScript pode induzir caso o desenvolvedor presuma que a verificação estática elimina a necessidade de validações em tempo de execução para dados externos (como entradas de formulários e respostas HTTP não confiáveis).

4.7 Confronto Teórico-Empírico e Síntese da Discussão

O Quadro 1 estabelece a síntese integrativa confrontando as premissas formuladas na literatura com os resultados empíricos apurados nesta pesquisa.

Quadro 1 – Confronto entre a literatura acadêmica e as evidências empíricas da pesquisa

Dimensão Teórica	Postulado da Literatura	Evidência Empírica da Pesquisa
Prevenção de bugs e robustez	O sistema de tipos estáticos pode antecipar a identificação de determinados erros (Richards; Nardelli; Vitek, 2017).	Percepção favorável. Média global de 4,17 para a afirmação de que JS gera mais bugs; 4,78 entre experientes para detecção precoce no TS.
Curva de aprendizado e transferência	O conhecimento prévio em JS pode gerar atrito conceitual na assimilação de TS (Shrestha et al., 2022).	Compatível. Iniciantes valorizam a flexibilidade do JS (4,29 em prototipagem) e relatam maior custo inicial da tipagem.
Qualidade de código versus complexidade	Aplicações em TS podem apresentar melhor qualidade percebida sem necessariamente reduzir todos os bugs (Bogner; Wagner, 2022).	Percepção compatível. Avaliação favorável de legibilidade (4,04), padronização (4,30) e rejeição da alegação de verbosidade excessiva (2,39).
Evolução e tendência de mercado	O suporte de frameworks e ferramentas favorece a adoção de TS (George, 2020; Stack Overflow, 2023).	Percepção favorável. Suporte de ecossistema (4,43), ferramentas/IDEs (4,56 para experientes) e expectativa de relevância futura (4,04).

Fonte: Elaborado pelo autor (2025).

Em suma, os resultados sugerem que a escolha entre JavaScript e TypeScript não é uma mera disputa estilística, mas uma decisão influenciada pelo horizonte temporal do projeto, pela experiência da equipe e pelas condições de integração. Na amostra analisada, o JavaScript foi associado à prototipagem e à introdução pedagógica, enquanto o TypeScript foi mais valorizado para manutenção, refatoração e coordenação em projetos maiores. Essa evidência não permite afirmar que uma das linguagens seja indispensável em todos os contextos.

5 Considerações Finais

Este trabalho teve como objetivo central investigar as percepções e os fatores que influenciam a adoção de JavaScript e TypeScript no desenvolvimento front-end, analisando comparativamente as diferenças entre desenvolvedores iniciantes e profissionais experientes. A coexistência dessas duas linguagens constitui um dos temas mais relevantes da engenharia de software contemporânea, moldando decisões arquiteturais, estratégias de contratação e práticas de governança de código em organizações de tecnologia de todos os portes.

Por meio de uma abordagem metodológica quantitativa e qualitativa baseada em levantamento de dados (*survey*), foi possível coletar e analisar as perspectivas de 23 desenvolvedores com diferentes níveis de maturidade profissional e vivência em projetos de variadas escalas. A seguir, apresentam-se as principais conclusões obtidas, as contribuições práticas do estudo, suas limitações e as recomendações para trabalhos futuros.

5.1 Síntese dos Resultados e Atendimento aos Objetivos

Os resultados empíricos obtidos contribuíram para responder aos objetivos específicos delineados na introdução deste trabalho:

1. **Análise comparativa das características técnicas:** A fundamentação teórica e as percepções coletadas indicam que o sistema de tipagem estática e opcional do TypeScript desloca parte da detecção de incompatibilidades para o tempo de compilação. Esse mecanismo pode apoiar a manutenção, mas não impede falhas de lógica, contratos incorretos, configuração inadequada ou problemas de runtime.
2. **Identificação dos benefícios e desafios do TypeScript:** Na amostra, os participantes associaram o TypeScript à manutenibilidade a longo prazo (média de 4,44 entre experientes), à detecção precoce de erros (média global de 4,17), à padronização em equipes grandes (média de 4,30) e ao apoio à refatoração. Também foram relatados custos de configuração, atrito com bibliotecas externas e necessidade de disciplina técnica contínua.
3. **Avaliação do impacto do nível de experiência:** Observou-se uma diferença de percepção entre os grupos. Iniciantes valorizaram a flexibilidade e a prototipagem rápida do JavaScript (média de 4,29), enquanto experientes relataram maior produtividade com TypeScript (média de 4,22, contra 2,93 dos iniciantes) e maior preferência por seu uso em novos projetos (média de 4,56). Como a amostra foi pequena e não probabilística, esses resultados não devem ser generalizados sem novas pesquisas.

5.2 Contribuições e Recomendações Práticas

A partir dos achados desta pesquisa, formulam-se diretrizes aplicadas destinadas tanto à gestão de projetos de software quanto à formação acadêmica:

5.2.1 Diretrizes para a Indústria e Gestores de Tecnologia

- **Projetos corporativos e de longo prazo:** Pode ser pertinente considerar TypeScript em aplicações empresariais, sistemas colaborativos e bases de código com previsão de sustentação contínua, avaliando equipe, legado, contratos externos, configuração e custo de migração. A adoção não deve ser tratada como obrigatória ou como garantia de ausência de regressões.
- **Prototipagem e MVPs:** O uso de JavaScript puro permanece altamente justificável em estágios exploratórios de produtos (*Minimum Viable Products*), validações conceituais imediatas ou scripts utilitários onde a velocidade inicial sobrepõe-se à rigidez estrutural.
- **Estratégias de Migração Gradual:** Equipes que planejam migrar sistemas legados de JavaScript para TypeScript devem prever um investimento deliberado em capacitação inicial e adotar estratégias incrementais (ativando modos permissivos antes de aplicar regras estritas de tipagem), evitando o uso indiscriminado do tipo `any`.

5.2.2 Diretrizes para o Ensino Acadêmico de Computação

- **Progressão Pedagógica:** Cursos de graduação e técnicos em computação e sistemas de informação devem manter o JavaScript no ciclo básico introdutório, dada sua menor barreira de entrada e facilidade didática (4, 17 geral).
- **Maturidade em Engenharia de Software:** Deve-se introduzir o TypeScript nas disciplinas avançadas de desenvolvimento web e arquitetura de software, preparando os futuros profissionais para as exigências reais do mercado de trabalho corporativo.

5.3 Limitações da Pesquisa

Como em qualquer estudo empírico, algumas limitações metodológicas devem ser consideradas na interpretação dos resultados:

1. **Tamanho da Amostra:** A pesquisa baseou-se em uma amostra de $N = 23$ participantes, selecionada por conveniência em comunidades de tecnologia e no ambiente acadêmico

do IFBA. Embora a amostra seja diversificada e tenha possibilitado análises comparativas consistentes, generalizações estatísticas para a totalidade da comunidade global de desenvolvedores devem ser realizadas com parcimônia.

2. **Natureza Autorreferida:** Os dados fundamentam-se nas percepções e autoavaliações dos próprios desenvolvedores, refletindo experiências subjetivas que podem ser influenciadas por preferências pessoais e tendências de mercado.

5.4 Sugestões para Trabalhos Futuros

Para dar continuidade e aprofundamento à linha de pesquisa explorada neste trabalho de conclusão de curso, sugerem-se as seguintes investigações:

1. **Estudo de Mineração de Repositórios:** Realizar uma análise quantitativa automatizada em larga escala de repositórios públicos (GitHub/GitLab) de empresas brasileiras, extraíndo métricas concretas de densidade de bugs, complexidade ciclomática e rotatividade de código (*code churn*) em projetos JS e TS.
2. **Impacto de Assistentes de Inteligência Artificial:** Investigar como o surgimento de ferramentas de inteligência artificial generativa e autocompletar inteligente (como GitHub Copilot e modelos baseados em LLM) impacta a curva de aprendizado de linguagens fortemente tipadas como o TypeScript.
3. **Estudo Longitudinal de Migração:** Acompanhar equipes de desenvolvimento durante o ciclo completo de migração de uma base legada em JavaScript para TypeScript, mensurando o tempo despendido, os tipos de erros encontrados e o impacto na velocidade de entrega (*lead time*).

Em conclusão, os resultados indicam que o TypeScript é percebido favoravelmente por profissionais experientes e em contextos de manutenção e colaboração, mas seus benefícios dependem de configuração, contratos confiáveis, validações de runtime e práticas de equipe. A escolha entre JavaScript e TypeScript deve ser compreendida como uma decisão contextual, na qual ganhos potenciais de longo prazo são avaliados em conjunto com o investimento inicial e as limitações do ecossistema.

Quadro 5 – Cronograma de atividades do TCC

Etapas	Mar/25	Abr/25	Mai/25	Jun/25	Jul/25	Ago/25	Set/25	Out/25	Nov/25	Dez/25	Jan/26	Fev/26	Mar/26
Escolha do tema	X												
Levantamento bibliográfico	X	X											
Definição do problema e objetivos		X	X										
Introdução			X	X									
Fundamentação Teórica				X	X								
Metodologia					X	X	X	X					
Formulário								X	X				
Conclusão										X	X		
Revisão geral do trabalho												X	X
Preparação da apresentação/defesa												X	X

Fonte: Elaborado pelo autor, 2025.

Referências

- ALVES, Nicolli S. R. *et al.* Identification and management of technical debt: A systematic mapping study. **Information and Software Technology**, v. 70, p. 100–121, 2016. DOI: [10.1016/j.infsof.2015.10.008](https://doi.org/10.1016/j.infsof.2015.10.008).
- BOGNER, Justus; WAGNER, Stefan. To type or not to type? A systematic comparison of the software quality of JavaScript and TypeScript applications on GitHub. *In: PROCEEDINGS of the 19th International Conference on Mining Software Repositories*. 2022. p. 552–563. DOI: [10.1145/3524842.3528454](https://doi.org/10.1145/3524842.3528454). Disponível em: <https://dl.acm.org/doi/abs/10.1145/3524842.3528454>.
- COUTINHO, Tamires L.; ANDRADE, Leandro; SANTANA, Cleber Jorge L. de. Soluções e desafios: Métodos ágeis na pesquisa colaborativa entre universidade e indústria. *In: ANAIS Estendidos do XIX Simpósio Brasileiro de Sistemas Colaborativos*. 2024. p. 247–251. DOI: [10.5753/sbsc_estendido.2024.238884](https://doi.org/10.5753/sbsc_estendido.2024.238884).
- CRISTIANI, Fernando; THIEMANN, Peter. Generation of TypeScript declaration files from JavaScript code. *In: PROCEEDINGS of the 18th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes*. 2021. p. 97–112. DOI: [10.1145/3475738.3480941](https://doi.org/10.1145/3475738.3480941).
- FELDTHAUS, Asger; MØLLER, Anders. Checking correctness of TypeScript interfaces for JavaScript libraries. *In: PROCEEDINGS of the 2014 ACM International Conference on Object Oriented Programming Systems Languages and Applications*. 2014. p. 1–16. DOI: [10.1145/2660193.2660215](https://doi.org/10.1145/2660193.2660215).
- GEORGE, Juliana. Adoption trends: Why developers are switching from JavaScript to TypeScript. **ResearchGate**, 2020. Disponível em: https://www.researchgate.net/publication/389688708_Adoption_Trends_Why_Developers_Are_Switching_from_JavaScript_to_TypeScript.
- GOMES, Felipe *et al.* Investigating the point of view of project management practitioners on technical debt: A study on Stack Exchange. **Journal of Software Engineering Research and Development**, v. 11, n. 1, 12:1–12:15, 2023. DOI: [10.5753/jserd.2023.3191](https://doi.org/10.5753/jserd.2023.3191).
- HOEFLICH, Joshua; FINDLER, Robert Bruce; SERRANO, Manuel. Highly illogical, Kirk: Spotting type mismatches in the large despite broken contracts, unsound types, and too many linters. **Proceedings of the ACM on Programming Languages**, v. 6, OOPSLA2, p. 479–504, 2022. DOI: [10.1145/3563305](https://doi.org/10.1145/3563305).
- JESSE, Kevin; DEVANBU, Premkumar T. ManyTypes4TypeScript: A comprehensive TypeScript dataset for sequence-based type inference. *In: PROCEEDINGS of the 19th*

International Conference on Mining Software Repositories. 2022. p. 294–298. DOI: [10.1145/3524842.3528507](https://doi.org/10.1145/3524842.3528507).

JESSE, Kevin; DEVANBU, Premkumar T.; SAWANT, Anand. Learning to predict user-defined types. **IEEE Transactions on Software Engineering**, v. 49, n. 4, p. 1508–1522, 2023. DOI: [10.1109/TSE.2022.3178945](https://doi.org/10.1109/TSE.2022.3178945).

MAKIMOV, Kirill. **A qualitative case study on using TypeScript as a JavaScript alternative in frontend web development in the industry**. 2022. Dissertação (Mestrado) – University of Tartu, Tartu. Disponível em: <https://dspace.ut.ee/bitstreams/76796c1b-8490-4a20-8925-99b1533eeaa9/download>.

MORORÓ, Jadson Faustino. **Um estudo comparativo entre JavaScript e TypeScript**. Sobral, 2024. Disponível em: https://repositorio.ufc.br/bitstream/riufc/78817/1/2024_tcc_jfmororo.pdf.

RICHARDS, Gregor; NARDELLI, Francesco Zappa; VITEK, Jan. To type or not to type: Quantifying detectable bugs in JavaScript. *In: PROCEEDINGS of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 2017. p. 758–769. Disponível em: <https://www.microsoft.com/en-us/research/wp-content/uploads/2017/09/gao2017javascript.pdf>.

SANTANA, Cleber Jorge Lira de; ALENCAR, Brenno de Mello; PRAZERES, Cássio V. Serafim. Reactive microservices for the Internet of Things: A case study in fog computing. *In: PROCEEDINGS of the 34th ACM/SIGAPP Symposium on Applied Computing*. 2019. p. 1243–1251. DOI: [10.1145/3297280.3297402](https://doi.org/10.1145/3297280.3297402).

SANTANA, Cleber Jorge Lira de; ANDRADE, Leandro *et al.* Increasing the availability of IoT applications with reactive microservices. **Service Oriented Computing and Applications**, v. 15, n. 2, p. 109–126, 2021. DOI: [10.1007/s11761-020-00308-8](https://doi.org/10.1007/s11761-020-00308-8).

SHRESTHA, Nischal *et al.* Here we go again: Why is it difficult for developers to learn another programming language? **Communications of the ACM**, 2022. Disponível em: <https://cacm.acm.org/research/here-we-go-again/>.

STACK OVERFLOW. **2023 Developer Survey**. 2023. Disponível em: <https://survey.stackoverflow.co/2023/>. Acesso em: 27 jul. 2025.

TANG, Tianyi; ALIMADADI, Saba; SUMNER, Nick. From logic to toolchains: An empirical study of bugs in the TypeScript ecosystem. *In: PROCEEDINGS of the 23rd International Conference on Mining Software Repositories*. 2026. p. 539–549. DOI: [10.1145/3793302.3793379](https://doi.org/10.1145/3793302.3793379). Disponível em: <https://arxiv.org/abs/2601.21186>.

WANG, Ziyuan; FANG, Yun; WANG, Nannan. An empirical study on bugs in TypeScript programming language. **Journal of Systems and Software**, v. 226, p. 112445, 2025. DOI: [10.1016/j.jss.2025.112445](https://doi.org/10.1016/j.jss.2025.112445).

ZANDONOTTO, Thomaz *et al.* How TypeScript compiler options influence code quality: A correlation study with Git repositories. **Journal of Software Engineering Research and Development**, v. 14, n. 1, p. 228–246, 2026. DOI: [10.5753/jserd.2026.5985](https://doi.org/10.5753/jserd.2026.5985).